

VCDSTIL Tutorial

Application Notes
February 17, 2011

Verigy Germany GmbH.
Herrenberger Strasse 130
71034 Boeblingen.

Contents

1	vcdstil Tutorial	1
1.1	Introduction	1
1.2	DUT Description - I8254	2
1.2.1	DUT overview	2
1.2.2	DUT pin definitions	2
1.2.3	DUT timing definitions	2
1.3	Basic vcdstil tutorial	3
1.4	Setup.py file content:	5
1.4.1	Explanation of commands	6
2	Advanced Tutorials	7
2.1	Using vcdstil with a spreadsheet (Excel/ Open office)	7
2.1.1	Run vcdstil with CSV (spreadsheet) file	7
2.1.2	Commands Explanation	11
2.2	How to use the Master file with ste	12
2.3	Multi (E)VCD files with master file	18
2.4	SSP tutorial	20
2.5	Input/Output timing definition tutorial	23

Chapter 1

vcdstil Tutorial

1.1 Introduction

1. **vcdstil concept**

The purpose of vcdstil software is to convert an event-based test (simulation results which are represented in (E)VCD formats) , to a cycle based format of STIL.

The inputs to vcdstil are design simulation files (Verilog VCD/EVCD dump files).

The Output is the STIL file format.

The vcdstil divides the (E)VCD source file into cycles, each different cycle is collected and translated to STIL waveform. The results include collection of waveforms vectors.

2. **Tutorial goal**

The purpose of this tutorial is to familiarize you with vcdstil software. After completing this tutorial you will be able to convert VCD files to STIL files .

For this tutorial we have chosen a VCD file because the signal direction is not defined in a VCD file and needs to be defined manually. For Inout pins I/O control signals (direction control) should be defined.

1.2 DUT Description - I8254

1.2.1 DUT overview

The Intel I8254 is a counter/timer device, it provides three independent 16-bit counters, each capable of handling clock inputs up to 10MHz.

For more information look at I8254 user manual.

1.2.2 DUT pin definitions

Below is the list of the device signals direction and I/O control signals as defined in the device specification:

Pin	Direction	I/O control
A0	Input	-
A1	Input	-
CLK0	Input	-
CLK1	Input	-
CLK2	Input	-
CS_	Input	-
D0	Inout	Controlled by D_dir
D1	Inout	Controlled by D_dir
D2	Inout	Controlled by D_dir
D3	Inout	Controlled by D_dir
D4	Inout	Controlled by D_dir
D5	Inout	Controlled by D_dir
D6	Inout	Controlled by D_dir
D7	Inout	Controlled by D_dir
GATE0	Input	-
GATE1	Input	-
GATE2	Input	-
OUT0	Output	-
OUT1	Output	-
OUT2	Output	-
RD_	Input	-
WR_	Input	-
D_dir	Input	-

Table 1.1: Device Direction definitions

D_dir is Internal I/O control pin for D0-D7 signals Note:

When D_dir I/O control signal value = 1 Inout signal direction is an Input.

When D_dir I/O control signal value = 0 Inout signal direction is an Output.

1.2.3 DUT timing definitions

The device operates at 2.5Mhz, which is why we select a tester cycle of 400ns.

All the rest of the definitions are generated automatically by vcdstil.

1.3 Basic vcdstl tutorial

Change directory to `./testcases/Extraction_Basic_Tutorial`

Under this directory you will find the following files:

1. **setup.py** - configuration file which include all the commands for vcdstl.
2. **WR_RD_Mode0.vcd** - dump file that was generated from the simulation of the DUT (I8254 in our case)

Run the conversion using the following command:

```
>vcdstl -setup setup.py
```

The generated log for this run:

```
Processing Setup...
Done [setup]

Analyzing Patterns...
  WR_RD_Mode0.vcd
Done [patterns]

Running Timing Optimizer...
Done [optimizer]

Checking Test Limits...
Done [test limits]

Generating STIL...
  Pattern:  ./stil/setup/WR_RD_Mode0.pat.stil
  Main File: ./stil/setup.stil
Done [stil]
```

As you can see vcdstl generates two output files in the following directories:

- **./stil/setup.stil:**
STIL waveform file
Contains STIL workspace blocks, also contains include statement for each generated pattern file.
- **./stil/setup/WR_RD_Mode0.pat.stil**
STIL pattern file
A separate pattern file is created from each input (E)VCD file. The file contains the STIL pattern generated from the corresponding input file.

You can look at these two files and see that each pin behaves like it's original behavior in the VCD file. For example, if you look only at pin 'CLK0' in the VCD file you will see it behave like:

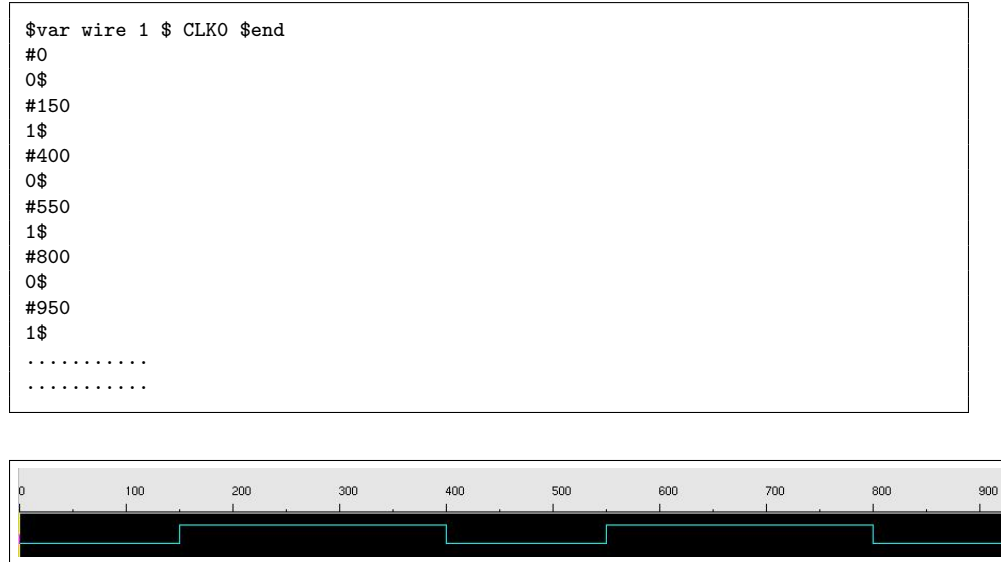
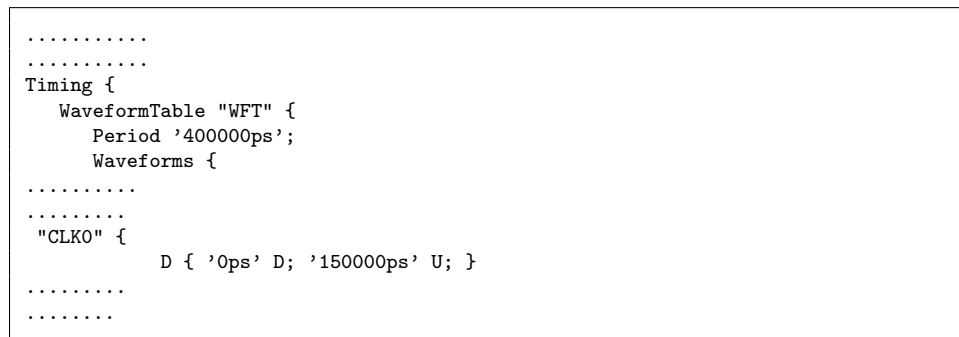


Figure 1.1: the source 'CLK0' as represented in VCD file

In the STIL waveform file (./stil/setup.stil) you will see



And in the STIL pattern file (./stil/setup/WR_RD_Mode0.pat.stil):



Figure 1.2: The result CLK0 as represented in STIL file

So we converted the 'CLK0' event-based behavior into behavior that is cycle-based.

1.4 Setup.py file content:

In the previous section we ran vcdstl using “-setup setup.py” command line parameters. These command line parameters pass the setup.py configuration file to vcdstl tool. Now let’s review the contents of the setup.py configuration file:

```
1
2 input_files = [ 'WR_RD_Mode0.vcd' ]
3
4 cycle_width = '400ns'
5
6 exclude_pins( pinlist = ['D_dir'] )
7
8 pin_attributes(
9     pinlist      = ['D?'],
10    type         = 'inout',
11    io_control    = 'in:D_dir=1,out:D_dir=0',
12 )
13
14 pin_attributes(
15     pinlist      = ['OUT?'],
16     type         = 'out',
17 )
18
19 pin_attributes(
20     pinlist      = ['A?','CS_','GATE?','RD_','WR_','CLK?'],
21     type         = 'in',
22 )
23
```

1.4.1 Explanation of commands

1. **In line 2:** `input_files = [ 'WR_RD_Mode0.vcd' ]` - The input VCD file
2. **In line 4:** `cycle_width = '400ns'` - The (E)VCD file Period
3. **In line 6:** `exclude_pins( pinlist = ['D_dir'] )` - Remove the D_dir pin since this signal is an internal I/O control pin for D0-D7 signals
4. **In line 8:** `pin_attributes( pinlist = ['D?'], type = 'inout', io_control = 'in:D_dir=1,out:D_dir=0', )`
 pin_attributes command use to define the timing and direction of the pins. We should define the pin attributes command for each pin in our design. Here we only define the direction of pins D0,D1...D7 to be inout when the D_dir signal controls the direction to be input or output (1 for input and 0 for output) .
5. **In line 14:** `pin_attributes( pinlist = ['OUT?'], type = 'out', )`
 We define the pins OUT0, OUT1 and OUT2 to be output .
6. **In line 19:** `pin_attributes( pinlist = ['A?','CS_','GATE?','RD_','WR_'], type = 'in', )`
 We define all the remaining pins to be input .

Chapter 2

Advanced Tutorials

Do these advanced tutorials only after you have completed the basic tutorial of the first chapter

2.1 Using vcdstil with a spreadsheet (Excel/ Open office)

In this tutorial we will use CSV files for editing the pins direction. The process and the results are the same as the first basic tutorial but here we define all the direction and timing of the pins in a CSV file.

What is a CSV File ?

The Comma Separated Value (CSV) file format is actually a text file format which can be edited as an Excel (Open Office) file.

Why use CSV (spreadsheet) Files ?

If you have a lot of pins in your design sometimes it easier to edit all of them in an Excel file.

2.1.1 Run vcdstil with CSV (spreadsheet) file

Change directory to `./testcases/Extraction_Tutorial_Using_CSV`

Under this directory you will find the following files:

1. `imported_pins_after_editing.csv`
2. `setup.py`
3. `WR_RD_Mode0.vcd`

Step 1: Fetch Pin information from VCD & Export it to a CSV file

First of all we will export all pins from the VCD file into CSV file format . Then we will open and setup the direction of the pins.

For generating the CSV file from WR_RD_Mode0.vcd run the following command:

```
> vcdstl -csv_export_pins imported_pins.csv WR_RD_Mode0.vcd
```

The generated log for this run:

```
1 Processing Setup...  
2 [csv_export_pins] Exporting to 'imported_pins.csv'.  
3 Done [setup]
```

Verify that imported_pins.csv file was created

Step 2: Update signals direction in a CSV file

Open the CSV file and update all signals direction according to the device spec.

In this tutorial we have done it for you, just compare the two files to see the difference. See 'imported_pins.csv' (the generated CSV file) and the 'imported_pins.after.editing.csv' (after editing) for reference.

	A	B	C	D	E	F
1	VS Pin Attributes	1				
2	Pin	Type	I/O Controls	Input Waveform	Output Waveform	StateMap
3	##HELP##	in/out/inout	in:pin=states,out:pin=states	time:state ... time:state <u>sp</u> =time	time:state ... time:state <u>sp</u> =time	map_name
4	D_dir	in				
5	A0	in				
6	A1	in				
7	CLK0	in				
8	CLK1	in				
9	CLK2	in				
10	CS_	in				
11	D0	in				
12	D1	in				
13	D2	in				
14	D3	in				
15	D4	in				
16	D5	in				
17	D6	in				
18	D7	in				
19	GATE0	in				
20	GATE1	in				
21	GATE2	in				
22	OUT0	in				
23	OUT1	in				
24	OUT2	in				
25	RD_	in				
26	WR_	in				

Figure 2.1: CSV file before editing

	A	B	C	D	E	F
1	VS Pin Attributes	1				
2	Pin	Type	I/O Controls	Input Waveform	Output Waveform	StateMap
3	##HELP##	in/out/ <u>inout</u>	in:pin=states	out:pin=states	time:state ... time:state <u>sp</u> =time	time:state ... time:state <u>sp</u> =time
4	A0	in				
5	A1	in				
6	CLK0	in				
7	CLK1	in				
8	CLK2	in				
9	CS_	in				
10	D0	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
11	D1	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
12	D2	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
13	D3	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
14	D4	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
15	D5	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
16	D6	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
17	D7	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
18	GATE0	in				
19	GATE1	in				
20	GATE2	in				
21	OUT0	out				
22	OUT1	out				
23	OUT2	out				
24	RD_	in				
25	WR	in				

Figure 2.2: CSV file after editing

We config the pins direction as defined in the device spec:

- D0-D7 signals will be inout and the I/O control signal for D0-D7 pins is D_dir pin.
- We configure the OUT0-2 to be an output.
- All other pins are configured to be an input.
- We have removed D_dir pin from the lists of the pins because it is only a control pin for D0-D7.

Step 3: Run vcdstil setup file

Run the conversion using the following command:

```
> vcdstil -setup setup.py
```

The generated log for this run:

```
Processing Setup...
[csv_import_pins] Importing from 'imported_pins_after_editing.csv'.
Done [setup]

Analyzing Patterns...
WR_RD_Mode0.vcd
Done [patterns]

Running Timing Optimizer...
Done [optimizer]

Checking Test Limits...
Done [test limits]

Generating STIL...
Pattern:  ./stil/setup/WR_RD_Mode0.pat.stil
Main File: ./stil/setup.stil
Done [stil]
```

Now let us examine closely the contents of the configuration file setup.py:

```
1
2 input_files = [ 'WR_RD_Mode0.vcd' ]
3
4 cycle_width = '400ns'
5
6 csv_import_pins( path= 'imported_pins_after_editing.csv')
7
```

2.1.2 Commands Explanation

1. **In line 2:** `input_files = [ 'WR_RD_Mode0.vcd' ]` - The input VCD file
2. **In line 4:** `cycle_width = '400ns'` - The STIL File Period
3. **In line 6:** `csv_import_pins( path= 'imported_pins_after_editing.csv')`
- Import the CSV file which we edited, it replaces the `pin_attributes` command that we used in the first basic tutorial.

2.2 How to use the Master file with ste

The object of this tutorial is to show how to use a STIL master file with vcdstil. You will find the tutorial under `./testcases/Master_Tutorial`. Under this directory you will find the following files:

1. `setup.py`
2. `WR_RD_Mode0.vcd`
3. `setup.stil`
4. `master_file/Master_STIL.stil`

The file `setup.stil` is the waveform file which was generated in the `Extraction_Basic_Tutorial` without the link to the pattern file in the end of the file.

We want to reduce the number of edges of `OUT0..OUT2`.

In order to do that we will do the following:

1. edit `setup.stil` with `ste` tool in order to mask `OUT0..OUT2`
2. Run `vcdstil` when importing the Master STIL file.
3. Compare the results

Step 1: Edit `setup.stil` with `ste` tool in order to mask `OUT0..OUT2`

Open the file `setup.stil` with `ste` by writing

```
> ste setup.stil
```

Now scroll down until you see the signals OUT0..OUT2

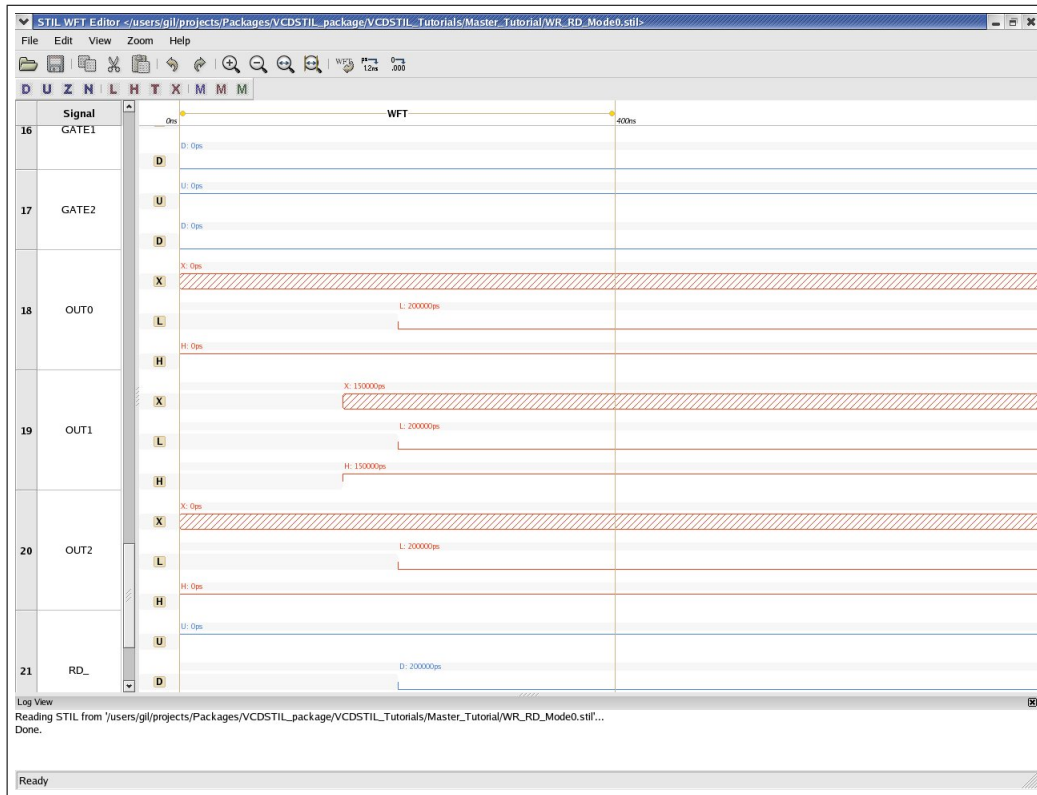


Figure 2.3:

Mark with a square the three signals OUT0..OUT2 in order to mask all the events before the last event and then push the red button marked M (mask output).

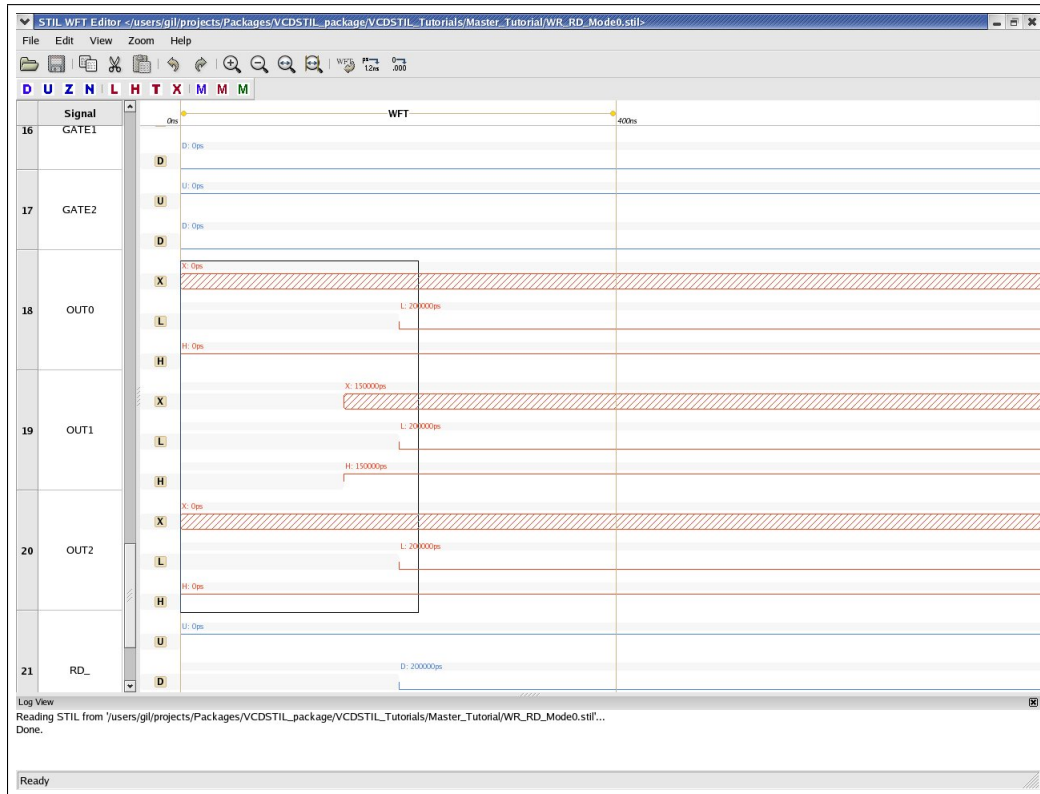


Figure 2.4:

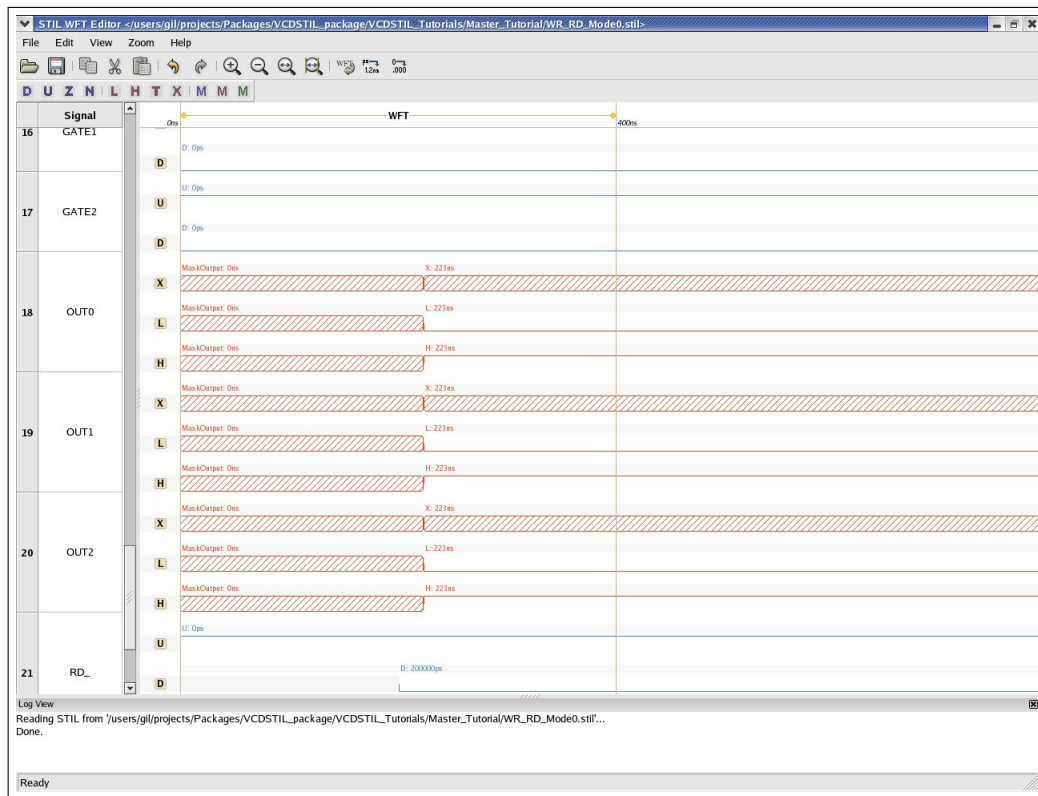


Figure 2.5:

Now choose from the menu 'File'-'Save As' and save as Master.STIL.stil. Don't save the file as setup.stil. We will use this file for comparing later results. Also, we have an example file for you. See the master file under the directory master_file. You can compare the results or copy that file and run the setup file with it.

Step 2: Run vcdstil with Master file

Type the following command in order to run vcdstil:

```
>vcdstil -setup setup.py
```

See the generated log:

```
Processing Setup...
[stil_import] Reading STIL Master File './Master_STIL.stil'.

Done [setup]

Analyzing Patterns...
WR_RD_Mode0.vcd
Done [patterns]

Running Timing Optimizer...
Done [optimizer]

Checking Test Limits...
Done [test limits]

Generating STIL...
Pattern: ./stil/setup/WR_RD_Mode0.pat.stil
Main File: ./stil/setup.stil
Done [stil]
```

Now let's examine closely the contents of the configuration file setup.py:

```
1  input_files = [ 'WR_RD_Mode0.vcd' ]
2
3  cycle_width = '400ns'
4
5  exclude_pins( pinlist = ['D_dir'] )
6
7  pin_attributes(
8      pinlist      = ['D?'],
9      type         = 'inout',
10     io_control    = 'in:D_dir=1,out:D_dir=0',
11 )
12
13 pin_attributes(
14     pinlist      = ['OUT?'],
15     type         = 'out',
16 )
17
18 pin_attributes(
19     pinlist      = ['A?','CS_','GATE?','RD_','WR_','CLK?'],
20     type         = 'in',
21 )
22
23 stil_import( path = './Master_STIL.stil' )
```

Explanation of Commands :

All the commands are identical to the basic tutorial just with the following command

1. In line 22: `stil_import( path = './Master_STIL.stil' )` - import the Master STIL file which we edited

Step 3: Compare the results

write:

```
diff setup.stil ./stil/setup.stil
```

```
134,136c134,136
<      X { '0ps' Z; '0ps' X; }
<      L { '200000ps' Z; '200000ps' L; }
<      H { '0ps' Z; '0ps' H; }
---
>      X { '223ns' X; }
>      L { '223ns' L; }
>      H { '223ns' H; }
139,141c139,141
<      X { '150000ps' Z; '150000ps' X; }
<      L { '200000ps' Z; '200000ps' L; }
<      H { '150000ps' Z; '150000ps' H; }
---
>      X { '223ns' X; }
>      L { '223ns' L; }
>      H { '223ns' H; }
144,146c144,146
<      X { '0ps' Z; '0ps' X; }
<      L { '200000ps' Z; '200000ps' L; }
<      H { '0ps' Z; '0ps' H; }
---
>      X { '223ns' X; }
>      L { '223ns' L; }
>      H { '223ns' H; }
```

You can see that the signals OUT0..OUT2 are masked as we defined in the master file. You can see these differences clearly using the command *tkdiff*.

In general, this process of using master files gives you control over the number of waveforms. This is because the results will contain fewer waveforms after we mask the unnecessary events.

2.3 Multi (E)VCD files with master file

This tutorial describes the option of using a couple of (E)VCD files as input to vcdstil with Master file .

It is recommend to do this tutorial after you have done the *How use Master file with ste* tutorial.

Change directory to `./testcases/Multi_VCD_With_Master/`

Under this directory you will find the following files:

1. `setup.py`
2. `WR_RD_Mode0.vcd`
3. `WR_RD_Mode1.vcd`
4. `WR_RD_Mode2.vcd`
5. `Master_STIL.stil`

Run the conversion using the following command:

```
> vcdstil -setup setup.py
```

The generated log for this run:

```
Processing Setup...
[stil_import] Reading STIL Master File './Master_STIL.stil'.
[stil_import] Warning: STIL Master File './Master_STIL.stil':
Done [setup]

Analyzing Patterns...
WR_RD_Mode0.vcd
WR_RD_Mode1.vcd
WR_RD_Mode2.vcd
Done [patterns]

Running Timing Optimizer...
Done [optimizer]

Checking Test Limits...
Done [test limits]

Generating STIL...
Pattern: ./stil/setup/WR_RD_Mode0.pat.stil
Pattern: ./stil/setup/WR_RD_Mode1.pat.stil
Pattern: ./stil/setup/WR_RD_Mode2.pat.stil
Main File: ./stil/setup.stil
Done [stil]
```

As you can see, we have four files with one workspace and pattern file for each VCD file:

If we look at the results we will see that all the pattern files are using the same timing (waveforms).

1. `./stil/setup.stil` - STIL workspace file
2. `./stil/setup/WR_RD_Mode0.pat.stil` - STIL pattern file for `WR_RD_Mode0.pat.stil`

3. `./stil/setup/WR_RD_Mode1.pat.stil` - STIL pattern file for WR_RD_Mode1.pat.stil

4. `./stil/setup/WR_RD_Mode2.pat.stil` - STIL pattern file for WR_RD_Mode2.pat.stil

Now let's examine closely the contents of the configuration file `setup.py`:

```
1  input_files = [ 'WR_RD_Mode0.vcd' , 'WR_RD_Mode1.vcd' , 'WR_RD_Mode2.vcd' ]
2
3  exclude_pins( pinlist = ['D_dir'] )
4
5  pin_attributes(
6      pinlist      = ['D?'],
7      type         = 'inout',
8      io_control   = 'in:D_dir=1,out:D_dir=0',
9  )
10
11 pin_attributes(
12     pinlist      = ['OUT?'],
13     type         = 'out',
14 )
15
16 pin_attributes(
17     pinlist      = ['A?','CS_','GATE?','RD_','WR_','CLK?'],
18     type         = 'in',
19 )
20
21 stil_import( path = './Master_STIL.stil' )
22
23 cycle_width = '400ns'
```

You can see we specified a couple of VCD files for input and we import the Master_STIL.stil file to vcdstil.

2.4 SSP tutorial

SSP (Smart Strobe Positioning) is a feature of vcdstil which automatically minimizes the number of waveforms and strobe positions created due to output waveform phases. The SSP feature only operates on output signals.

In this tutorial we show the SSP feature in vcdstil.

Change directory to `./testcases/SSP-Tutorial/`

Under this directory you will find the following files:

1. `setup.py`
2. `WR_RD_Mode0.vcd`

At first we run the conversion without SSP feature Run the following command:

```
>vcdstil -setup setup.py
```

See the generated log:

```
Processing Setup...
Done [setup]

Analyzing Patterns...
  WR_RD_Mode0.vcd
Done [patterns]

Running Timing Optimizer...
Done [optimizer]

Checking Test Limits...
Done [test limits]

Generating STIL...
  Pattern:  ./stil/setup/WR_RD_Mode0.pat.stil
  Main File: ./stil/setup.stil
Done [stil]
```

Copy the main file which generated to `STIL_without_SSP.stil`.

Run the following command:

```
> cp ./stil/setup.stil STIL_without_SSP.stil
```

Open the `setup.py` file.

```

input_files = [ 'WR_RD_Mode0.vcd' ]

cycle_width = '400ns'

exclude_pins( pinlist = ['D_dir'] )

pin_attributes(
    pinlist      = ['D?'],
    type         = 'inout',
    io_control   = 'in:D_dir=1,out:D_dir=0',
)

pin_attributes(
    pinlist      = ['OUT?'],
    type         = 'out',
)

pin_attributes(
    pinlist      = ['A?','CS_','GATE?','RD_','WR_','CLK?'],
    type         = 'in',
)

#ssp(pinlist    = ['OUT?'] )

```

As you can see the last command of SSP is in comment format ('#'). Remove this comment and run it again using the following command:

```
>vcdstil -setup setup.py
```

Now all that is left to do is to compare the files. Run the following command:

```
>diff STIL_without_SSP.stil ./stil/setup.stil
```

```

135,136c135,136
<          L { '200000ps' Z; '200000ps' L; }
<          H { '0ps' Z; '0ps' H; }
---
>          L { '300000ps' Z; '300000ps' L; }
>          H { '300000ps' Z; '300000ps' H; }
140,141c140,141
<          L { '200000ps' Z; '200000ps' L; }
<          H { '150000ps' Z; '150000ps' H; }
---
>          L { '300000ps' Z; '300000ps' L; }
>          H { '300000ps' Z; '300000ps' H; }
145,146c145,146
<          L { '200000ps' Z; '200000ps' L; }
<          H { '0ps' Z; '0ps' H; }
---
>          L { '300000ps' Z; '300000ps' L; }
>          H { '300000ps' Z; '300000ps' H; }

```

You can see that the SSP has chosen one sampling point for each OUT signal. You can see these differences clearly with the command *tkdiff*



Figure 2.6: The result STIL file without SSP feature



Figure 2.7: The result STIL file with SSP feature

2.5 Input/Output timing definition tutorial

In this Tutorial we will see how to config manually the timing for pins in vcdstil. We want the STIL timing file will behave like this diagram:

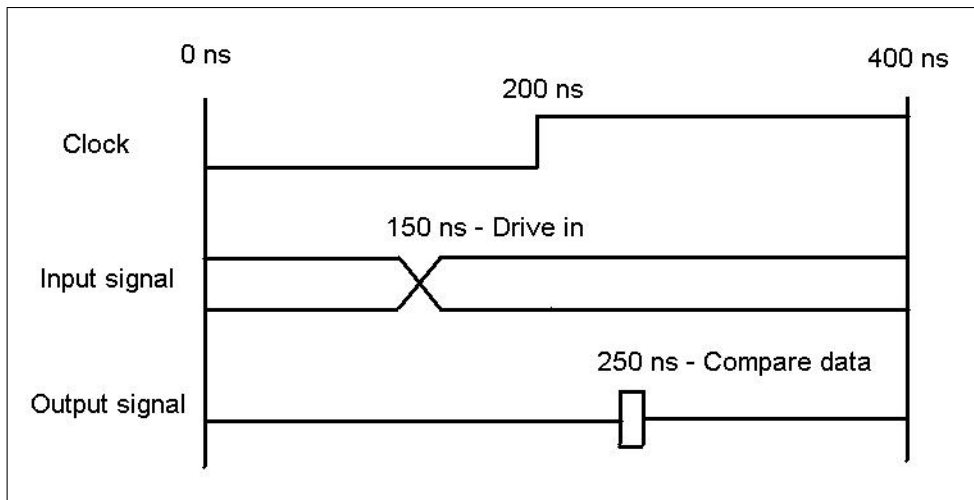


Figure 2.8: Diagram of timing for input/output and clock signals

Explanation:

1. The device operates at 2.5Mhz. As a result we select the tester cycle to be 400ns. Also, we will set the clock to change at 200ns.
2. For input signals we will drive the sample data at 150ns which is 50ns before the posedge of the clock.
3. For output signals we will compare them at 250ns which is 50ns after the posedge of the clock.

Change directory to `./testcases/Input_Output_Timing_Definition_Tutorial/`

Under this directory you will find the following files:

1. `setup.py`
2. `WR_RD_Mode0.vcd`

Look at the setup file :

```

1  input_files = [ 'WR_RD_Mode0.vcd' ]
2
3  cycle_width = '400ns'
4
5  exclude_pins( pinlist = ['D_dir'] )
6
7  pin_attributes(
8      pinlist      = ['CLK?'],
9      type         = 'in',
10     input_waveform = '0ns:D 200ns:U'
11 )
12
13 pin_attributes(
14     pinlist      = ['D?'],
15     type         = 'inout',
16     io_control   = 'in:D_dir=1,out:D_dir=0',
17     input_waveform = '150ns:S',
18     output_waveform = '250ns:S '
19 )
20
21 pin_attributes(
22     pinlist      = ['OUT?'],
23     type         = 'out',
24     output_waveform = '250ns:S'
25 )
26
27 pin_attributes(
28     pinlist      = ['A?','CS_','GATE?','RD_','WR_'],
29     type         = 'in',
30     input_waveform = '150ns:S'
31 )
32

```

As you can see we set each pin the timing which defined in the timing spec.

1. In line 10 : for the all the clock pins we defined to drive low at 0ns and drive high at 200ns.
2. In line 17-18: for all the inout pins we defined for input state drive the sample of the VCD at 150ns and for output state compare the sample of the VCD at 250ns.
3. In line 24 : for all the output pins compare the sample of the VCD at 250ns.
4. In line 30 : for all the input pins drive the sample of the VCD at 150ns.

Run the following command:

```
>vcdstil -setup setup.py
```

See the generated log:

```

Processing Setup...
Done [setup]

Analyzing Patterns...
  WR_RD_Mode0.vcd
Done [patterns]

Running Timing Optimizer...
Done [optimizer]

Checking Test Limits...
Done [test limits]

Generating STIL...
  Pattern:  ./stil/setup/WR_RD_Mode0.pat.stil
  Main File: ./stil/setup.stil
Done [stil]

```

Now look at the generated STIL file `./stil/setup.stil` and you will see the following content:

```

Timing {
  WaveformTable "WFT" {
    Period '400000ps';
    Waveforms {
      "A0" {
        N { '150000ps' N; }
        U { '150000ps' U; }
        D { '150000ps' D; }
      }
      .....
      "D0" {
        N { '150000ps' N; }
        D { '150000ps' D; }
        U { '150000ps' U; }
        L { '250000ps' L; }
        T { '250000ps' T; }
        H { '250000ps' H; }
      }
      .....
      "OUT0" {
        X { '250000ps' X; }
        L { '250000ps' L; }
        H { '250000ps' H; }
      }
    }
  }
  .....
}

```

You can see that the STIL file timing is drive/compare at the points we have defined.

e.g: Here we have defined the timing definition for all the pins, of course you can define timing for a couple of pins...